INFERLAB

Production Prompt Rulebook

A step-by-step AI coding playbook: from an empty repository to a publicly deployed, zero-cloud-spend LLM inference benchmarking platform.

PROJECT	STACK / PRINCIPLE
Frontend	React, TypeScript, Vite, Tailwind, shadcn/ui, TanStack Query, Plotly
Backend	Python, FastAPI, Pydantic, SQLAlchemy, PostgreSQL
ML core	Your own Python benchmarks, profilers, evaluation and experiment logic
Inference	Ollama + llama.cpp on your RTX 4060 8 GB GPU
Public deployment	Cloudflare Pages + published benchmark snapshots + optional Neon free database
Private runner	Local laptop; secured access; no unauthenticated public GPU execution

Prepared for Amit Kumar | October 2026 | Version 1.0

Reality check: no-fee hosting is achievable for a static read-only portfolio within free-tier quotas. It does not provide free continuous GPU compute, a domain, electricity, permanent uptime or unlimited database usage. Free-tier terms can change.

How to use this rulebook

Treat each numbered prompt as a separate AI coding session or commit. Paste the Master Rules first in your coding assistant, then run prompts in order. Do not skip acceptance gates. For experiments, you implement the Python logic yourself and use AI for design critique, tests, documentation and frontend code.

- Before each prompt: checkout or create a focused branch; commit the last working state.
- Provide the AI with relevant files, current repository tree and error output; do not paste secrets.
- Require a plan and file-change list before code for security-sensitive changes.
- Review the diff and run tests; ask for corrections before moving to the next prompt.
- Never claim a benchmark gain without a fair baseline and reproducible measurements.
- Push only synthetic/public evaluation data and safe example configurations.

Recommended delivery sequence

Stage	Prompts	Exit criteria
Foundation	00-04	Monorepo, tooling, contracts and mocked UI
ML core	05-12	Accurate reproducible Python benchmark engine
Backend + UI	13-18	Safe experiment API and functional interface
Quality + release	19-25	Tests, security, snapshots, cloud hosting and release

Master Rules - paste at the beginning of every AI coding session

```
You are my senior software engineering pair for InferLab. I am personally implementing the Python ML/inference benchmark logic to learn ML systems. You may generate React/TypeScript frontend, routine API scaffolding, tests, docs and refactors. Do NOT implement the core benchmark algorithms for me unless I explicitly ask for a small illustrative example. For each task: (1) inspect only relevant project files, (2) summarize existing behavior, (3) propose a minimal plan, (4) identify exact files to create/change, (5) implement only the requested scope, (6) add focused tests, (7) provide verification commands and expected results, (8) describe security, resource and data-loss risks. Never invent benchmark measurements, guarantee free tiers, store secrets in Git, silently add paid services, expose an unauthenticated GPU endpoint, or broaden work into unrelated features. Prefer readable modular code, explicit types, and reproducibility. Ask a blocking question when a material decision is missing.
```

Technical non-negotiables

- Local NVIDIA GPU must never be a free public anonymous inference service.
- The public site must render previous benchmark results when the worker is offline.
- Frontend must not connect directly to a privileged local inference server.
- All public API endpoints need input validation; administrative actions need authentication, authorization, rate limits and audit records.
- A database queue is acceptable only with one active worker, leases, retries and idempotent completion; avoid a premature Redis dependency.
- Do not run arbitrary user-supplied shell commands, file paths, URLs or model repositories.

- Hold the model revision, workload, output budget, sampling and hardware conditions constant in fair comparisons.

PROMPT 00 | Product contract and scope

Stage: Foundation. Run when: Ask AI to write specification only, no code.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Draft a concise product requirements document for InferLab. Users: public viewers and one authenticated admin. Public viewers can read published benchmark results and method explanations; only the admin can submit local experiments. Define MVP vs stretch, screen list, public/private API endpoints, worker online/offline behavior, free hosting constraints, assumptions, security threats, and out-of-scope features. Produce docs/PRD.md, docs/ARCHITECTURE.md, docs/ADR-001-zero-cost.md and a measurable definition-of-done. No implementation.
```

Expected output: PRD, architecture decisions, risk register

Gate before next prompt: Product scope explicitly excludes public unrestricted GPU execution.

PROMPT 01 | Repository and monorepo skeleton

Stage: Foundation. Run when: Create Git repo, Node and Python environments.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Set up an InferLab monorepo with apps/web (Vite React TypeScript), apps/api (FastAPI), packages/inferlab_core (installable Python package), worker, experiments, tests, docs and .github/workflows. Add .gitignore, README, sample environment files, lint configuration, Python pyproject.toml with pinned dependency ranges, npm lockfile, and setup scripts for Windows PowerShell. Do not add cloud keys, paid services, ML algorithms, or Docker yet. Show commands to run each app.
```

Expected output: Buildable skeleton and setup docs

Gate before next prompt: Fresh clone installs frontend and Python tooling without secrets.

PROMPT 02 | Engineering conventions and CI guardrails

Stage: Foundation. Run when: Establish standards before feature work.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Configure TypeScript strict mode, ESLint, formatting, pytest, Ruff, mypy or pyright, pre-commit optional hooks, and GitHub Actions for CPU-only lint and unit tests. Create CONTRIBUTING.md defining conventional commits, PR checklist, secret-scanning guidance, testing expectations, and the boundary around my Python benchmark logic. Avoid GPU jobs on hosted CI. Ensure CI passes on an empty implementation.
```

Expected output: Static checks and working CI

Gate before next prompt: CI does not need a GPU or paid runner.

PROMPT 03 | Frontend design system

Stage: Foundation. Run when: Generate UI foundation, not the whole app.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Build an elegant engineering dashboard design system using React, TypeScript, Tailwind and shadcn/ui. Design tokens: restrained slate/navy, accessible contrast, dense professional tables,
```

```
responsive navigation. Create AppShell, Sidebar, Header, StatusBadge, MetricCard, EmptyState, ErrorState and Skeleton. Add routes for Overview, Experiments, Compare, Models, Reports and Methodology. Use explicit mock data only; label it DEMO DATA. No real backend integration yet. Include accessible keyboard navigation and mobile layouts.
```

Expected output: Responsive clickable UI shell

Gate before next prompt: No fake measurement is presented as real.

PROMPT 04 | Typed API contracts and mock layer

Stage: Foundation. Run when: Make frontend/backend coordinate cleanly.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
From docs/PRD.md, define Pydantic request/response schemas and matching TypeScript types or generated OpenAPI client types. Cover engine, model, experiment config, job state, run metrics, summary, published report and worker heartbeat. Use mock API fixtures and TanStack Query hooks. Explicitly model unavailable, pending, running, complete, failed and cancelled states. Do not implement benchmark math.
```

Expected output: Validated typed contracts and mocked integration

Gate before next prompt: Schema/type tests pass and offline UI works.

PROMPT 05 | Python inference science starter

Stage: ML core. Run when: This prompt teaches you, not writes the algorithm.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Act as an ML systems tutor. Teach me autoregressive generation, prefill, decode, TTFT, TPOT, KV cache and memory model with small Python exercises. Propose function signatures and test cases for inferlab_core/benchmarking/metrics.py but DO NOT write its implementation. Ask me to implement. After I share code, review correctness and edge cases. Give official reference links.
```

Expected output: My handwritten metrics implementation

Gate before next prompt: Correct units, timestamps and no division-by-zero bugs.

PROMPT 06 | Ollama transport adapter

Stage: ML core. Run when: AI can scaffold transport; you own measurements.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Create a minimal Ollama HTTP transport adapter behind a common EngineProtocol. Support generate, stream_generate, health_check and model info. Use HTTPX timeouts and typed errors. Handle Ollama duration fields as nanoseconds, streaming NDJSON and cancellation. Keep benchmark timing and calculations as TODOs for me. Write adapter tests using mock responses. Do not pull models automatically.
```

Expected output: Working Ollama adapter

Gate before next prompt: Integration test works with a local small model.

PROMPT 07 | Measurement methodology review

Stage: ML core. Run when: Explicit warm/cold conditions.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Review my design for benchmark_runner.py. Provide a written algorithm, test matrix and function stubs only. It must separate cold-start/model loading from warmed inference, use monotonic clocks, record client TTFT from streamed first token, measure total request latency, record input/output tokens, and repeat runs. Flag risks including CUDA synchronization, network noise, output-length mismatch, clock granularity, and model keep-alive effects. I will implement the algorithm.
```

Expected output: Measurement methodology and unit tests

Gate before next prompt: Warm and cold metrics never mix; TTFT actually uses streaming.

PROMPT 08 | Experiment configuration and results schema

Stage: ML core. Run when: All parameters reproducible.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Define a Pydantic ExperimentConfig and Result schema with stable experiment ID, model ID/revision/checksum, engine version/build, quantization, context, prompt ID, input tokens, max output tokens, sampling parameters, GPU settings, seed if supported, warmups, repetitions, concurrency, run timestamp, device/driver and errors. Provide YAML fixtures and validation tests. Do not implement benchmark execution.
```

Expected output: Strict experiment configuration

Gate before next prompt: Invalid sizes, unknown engines and unsafe paths are rejected.

PROMPT 09 | llama.cpp backend adapter

Stage: ML core. Run when: Ensure equivalent workload semantics.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Build a llama.cpp server adapter implementing the shared EngineProtocol. Use locally installed llama-server and its documented HTTP API, typed responses, health checks, streaming output, errors and timeouts. Expose configuration for GPU layers and context. Do not attempt automatic binary downloads or runtime shell command execution from public requests. Write parity tests with the Ollama adapter.
```

Expected output: Second working engine adapter

Gate before next prompt: Same test harness can run both engines.

PROMPT 10 | NVIDIA GPU telemetry API

Stage: ML core. Run when: You write the sampler.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Teach me how to measure GPU used memory, free memory, utilization, temperature and sampling timestamp through NVIDIA NVML. Propose interfaces and mockable tests for gpu_profiler.py but leave sampling implementation to me. Explain baseline subtraction, sampling interval, driver/tool errors, process-vs-device memory and why NVML may miss very short peaks.
```

Expected output: My NVML telemetry module

Gate before next prompt: Samples captured with units and timestamps, no invented values.

PROMPT 11 | Quantization and offloading experiment planner

Stage: ML core. Run when: Compare like for like.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Design a matrix to compare Q4_K_M, Q5_K_M, Q8_0 and FP16 when feasible for the SAME exact model revision on RTX 4060 8GB. Vary context and llama.cpp GPU layers in separate controlled trials. Specify matching prompts, output budgets, cooldown, repetition and quality checks. Generate only YAML configurations, an experiment protocol and tests for plan validation. I will execute and analyze results.
```

Expected output: Versioned experiment configs

Gate before next prompt: One factor varies at a time where appropriate.

PROMPT 12 | Concurrency and quality evaluation tutoring

Stage: ML core. Run when: You implement async load and scoring.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Review my concurrency_runner.py and evaluation.py design. Teach client-side load generation, controlled arrival rates, queueing, p50/p95, request errors, aggregate output throughput, and deterministic task-specific evaluation. Suggest tests for 1/2/4/8 clients and public/synthetic question sets. Do not write the benchmarking algorithm. Critique unfair model comparisons and fragile statistics.
```

Expected output: Your concurrency and evaluation modules

Gate before next prompt: Quality and speed reported together with sample sizes.

PROMPT 13 | Database schema and migrations

Stage: Backend. Run when: Implement persistent results safely.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Create SQLAlchemy models and Alembic migrations for ModelRegistry, Experiment, Job, RunMetric, PublishedReport and AuditEvent. Develop against SQLite and support PostgreSQL by environment URL. Store measurements as typed units; keep bulky trace artifacts outside database. Add indexes and constraints, migration tests, retention policy notes. Avoid recording raw personal user prompts by default.
```

Expected output: Portable database schema

Gate before next prompt: SQLite and PostgreSQL schemas migrate cleanly.

PROMPT 14 | Experiment API and job lifecycle

Stage: Backend. Run when: Admin may queue but not run shell commands.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Create FastAPI endpoints for public reports/metadata and admin experiment CRUD. State machine: queued, leased, running, completed, failed, cancelled. Add Pydantic validation, pagination, idempotency keys, errors and audit logs. Use dependency injection for core runners. Do not expose arbitrary shell execution, file path input, model downloads or unbounded resource settings. Add API tests.
```

Expected output: Working REST API

Gate before next prompt: Bad configs are rejected; only admin can queue jobs.

PROMPT 15 | Single-GPU worker and lease safety

Stage: Backend. Run when: Resource scheduling with bounds.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Implement a private Python worker that polls authorized queued experiment records. Enforce single active GPU job, bounded run time, bounded max tokens, fixed allowed model registry, lease heartbeat, cancellation, retry limits and idempotent result writes. Do not make the worker publicly addressable. Handle laptop shutdown and recovery without marking incomplete data successful. Supply failure/restart tests.
```

Expected output: Resilient private worker

Gate before next prompt: No duplicate GPU jobs; interrupted jobs recover safely.

PROMPT 16 | Authentication and authorization

Stage: Backend. Run when: Single-admin first.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Design and implement the simplest secure authentication for a single admin. Compare a managed free-tier auth provider vs self-hosting and choose based on current availability. Require server-side verified identity and authorization on every mutation. Add secure session/cookie configuration, CSRF mitigation when applicable, CORS allowlist, rate limits, secrets management and audit logs. Avoid custom password cryptography. Public result endpoints require no login.
```

Expected output: Protected admin API

Gate before next prompt: Anonymous caller cannot submit, alter or delete experiments.

PROMPT 17 | React experiments dashboard

Stage: Frontend. Run when: Use real API contracts.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Connect the existing React UI to FastAPI through typed client and TanStack Query. Build experiment list, details, config editor and job-state progress. Validate form constraints; show worker offline, empty and errors. Never claim a job succeeded before confirmed server status. Use semantic forms, clear units and helpful failure messages. Keep public browsing read-only and admin controls protected.
```

Expected output: Experiment management frontend

Gate before next prompt: Contract tests and browser user flows pass.

PROMPT 18 | Charts, comparisons and reports

Stage: Frontend. Run when: Accurate analytical presentation.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Build Plotly views for decode TPS, TTFT, p95 latency, aggregate throughput, peak VRAM and quality. Show model revision, quantization, prompt distribution, sample count and hardware context beside every comparison. Create CSV/JSON downloads and a printable report. Use only measured results or visibly tagged demo fixtures. Never imply causation from an uncontrolled comparison.
```

Expected output: Transparent scientific dashboard

Gate before next prompt: Every chart can trace back to underlying run IDs.

PROMPT 19 | Snapshot publishing pipeline

Stage: Deployment. Run when: Static site always available.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Build a publish step that transforms reviewed benchmark results into versioned, sanitized static JSON artifacts for the public frontend. Add schema validation, checksums and a manifest. Exclude credentials, private prompts, local paths and raw logs. Make the React app read published snapshots when the worker/backend is offline; label snapshot date prominently. No automatic publish before validation.
```

Expected output: Offline-capable public dashboard

Gate before next prompt: Public site loads even with local GPU and API powered off.

PROMPT 20 | Test suite and security gates

Stage: Quality. Run when: Block unsafe releases.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Add pytest unit/integration tests, Vitest UI tests, Playwright smoke tests, API authorization tests, schema contract tests and synthetic benchmark regression tests. Add checks for CORS, injection, path traversal, unbounded generation, SSRF-style input, job replay and accidental secret disclosure. Keep all CI tests CPU-only, deterministic and free. Output a release gate checklist.
```

Expected output: Passing reliable automated checks

Gate before next prompt: Critical security and functional tests pass.

PROMPT 21 | Docker and local runbooks

Stage: Quality. Run when: Optional reproducible containers.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Provide Dockerfiles for web build and FastAPI/worker services, Docker Compose for local database and app, health checks and environment templates. Preserve ability to run native Windows CUDA inference outside containers if CUDA container setup is impractical. Never bundle model weights or secrets into images. Explain host GPU caveats and verify startup/shutdown behavior.
```

Expected output: Reproducible local stack

Gate before next prompt: One documented startup path works on developer machine.

PROMPT 22 | Cloudflare Pages deployment

Stage: Deployment. Run when: Deploy static React build.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Write exact steps to deploy apps/web to Cloudflare Pages using a GitHub repository and free tier. Configure build output, environment variables for public read-only API/snapshot base URL, HTTPS, SPA routing and caching rules. No sensitive key may appear in VITE_ variables. Add a deploy smoke-test checklist. Do not create a paid subscription or custom domain by default.
```

Expected output: Live public read-only frontend

Gate before next prompt: Public URL loads on mobile and worker-offline state.

PROMPT 23 | Neon data and private worker connectivity

Stage: Deployment. Run when: Choose minimal service exposure.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Assess whether the static snapshot MVP even needs a cloud database. If yes, document Neon free PostgreSQL provisioning, SSL connection strings, least privilege, migration, quotas, backup/export, connection limits and safe pooling. Show an option where the local worker pushes sanitized reports to a private publish process without exposing FastAPI publicly. Cloudflare Tunnel may be used for admin-only access after auth and origin restrictions. No open public inference route.
```

Expected output: Data deployed within free quotas

Gate before next prompt: No unrestricted internet path to local GPU.

PROMPT 24 | Operational monitoring and incident plan

Stage: Deployment. Run when: Be transparent about uptime.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Implement basic health, worker heartbeat, job audit logs, bounded retention, error tracking and a manual stop switch. Show Worker Offline when laptop sleeps, network disconnects or worker loses lease. Produce a runbook for recovering failed jobs, rotating credentials, restoring DB from exports and removing a compromised tunnel. Avoid introducing paid telemetry products.
```

Expected output: Operate and recover safely

Gate before next prompt: Turning laptop off never breaks public read-only pages.

PROMPT 25 | Launch, demonstration and portfolio case study

Stage: Release. Run when: Explain what you personally built.

COPY / PASTE INTO YOUR CODING ASSISTANT:

```
Produce final README and architecture diagrams, reproducible benchmark protocol, system limitations, cost table, security model, measured performance analysis, screenshots, 3-minute demo script and honest resume bullets. Separate my own Python algorithms from third-party libraries and AI-generated UI code. Never invent speedups or indicate 24/7 GPU hosting. Review the full project against the original PRD and create a concise v1.0 release checklist.
```

Expected output: Launch-ready project case study

Gate before next prompt: Another engineer can reproduce at least one public benchmark.

Reusable prompts for every coding session

Plan-only preflight

```
Inspect the relevant files and summarize the current state. Provide a 5-step implementation plan, exact file list, risks and acceptance criteria. Make NO changes until I approve.
```

Review Python benchmark code

```
Review my Python implementation for numerical correctness, units, timing, fairness, race conditions, input bounds, and reproducibility. List concrete issues ranked by severity. Do not rewrite it; propose minimal patches and tests.
```

Debug a failure

```
Here is the exact command, full error, expected behavior, and relevant source. Identify the most likely root cause, propose the smallest reversible fix, show verification, and state any remaining uncertainty. Do not refactor unrelated files.
```

Test generation

```
For these specific functions, generate pytest tests for normal cases, edge cases, failures and numerical units. Do not modify the implementation. Explain what each test protects.
```

Frontend visual polish

```
Improve only this React page to professional SaaS-level hierarchy, spacing, typography, responsive layout, skeleton/error/empty states and keyboard accessibility. Keep data contracts and all business logic unchanged. Do not fabricate metrics.
```

Security review

```
Threat-model this change as though a public visitor is malicious. Check auth, authorization, request limits, path handling, secrets, CORS, SSRF, DoS, job replay, unsafe deserialization and tunnel exposure. Return findings and concrete patches.
```

Pre-merge diff audit

```
Summarize the git diff, altered contracts, migration risks, new dependencies, test results and unaddressed TODOs. Flag unrequested modifications and any paid service, credential exposure or fake metric.
```

Launch readiness checklist

- [ ] Frontend builds and is responsive on desktop and mobile.
- [ ] Public read-only pages work with local API and GPU worker entirely offline.
- [ ] Only authenticated admin can queue/cancel experiments.
- [ ] No arbitrary model URL, filesystem path or shell execution is accepted.
- [ ] Job limits, timeouts, cancellation and restart recovery are tested.
- [ ] Models are pinned by revision/checksum and benchmarks retain settings.
- [ ] Published metrics derive from actual saved runs; demo figures are labeled.
- [ ] Privacy and licensing checks pass for models, prompts and datasets.
- [ ] No secrets are exposed in frontend bundles, logs or Git history.
- [ ] Deployment quotas, off-switch and recovery runbooks are documented.
- [ ] A stranger can reproduce one experiment from the published report.

Zero-cost bill of materials and limitations

Component	Recommended zero-fee strategy	Limitation
Frontend	Cloudflare Pages free tier	Build and bandwidth quotas; availability subject to plan terms
Backend / worker	Local FastAPI + laptop GPU	Only available while laptop is on; electricity and internet not free
Read-only report data	Versioned static JSON on Pages	Publish workflow required; no live updates when offline
Optional DB	Neon free PostgreSQL	Storage/compute quotas; monitor limits
Authentication	Single-admin managed free tier or private authenticated channel	Check current quotas and identity-security configuration
Public domain	Provider-issued free subdomain	Custom domain registration is normally a cost
CI/CD	GitHub Actions free allocation	Hosted minutes and usage limits apply

Cost estimates assume a small non-commercial portfolio and current published free quotas. A production-grade architecture does not imply an SLA or high availability at zero spend.

Official implementation references

Cloudflare Pages limits https://developers.cloudflare.com/pages/platform/limits/

Cloudflare Tunnel https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/

Neon free tier https://neon.com/pricing

FastAPI https://fastapi.tiangolo.com/

React https://react.dev/

Vite https://vite.dev/

TanStack Query https://tanstack.com/query/latest

shadcn/ui https://ui.shadcn.com/

Ollama API https://docs.ollama.com/api

llama.cpp https://github.com/ggml-org/llama.cpp

PyTorch Profiler https://docs.pytorch.org/tutorials/beginner/profiler.html

Locust https://docs.locust.io/

Playwright https://playwright.dev/

GitHub Actions https://docs.github.com/en/actions